Chapter 2

Relational Data Model

Unit 2.2: Concept Of RDBMS, E. F. Codd's Rule for RDBMS, Key concepts – Candidate Key, Primary Key, Foreign Key

Prepared By

Mr. Mohammad Ali

Lecturer (Selection Grade)

M. H. Saboo Siddik Polytechnic, Mumbai

Learning Outcomes

The Learners will be able to:

1. Explain the concept of RDBMS.

Codd’s Rule

- E.F. Codd was a Computer Scientist at IBM who invented **Relational model** for Database management in 1970.
- Based on relational model, **Relation database management system(RDBMS)** was created.
- In 1985, Dr. Codd proposed 13(0 - 12) rules popularly known as **Codd's 12 rules** to test DBMS's concept against his relational model.
- Codd's rule actually define what quality a DBMS requires in order to become a Relational Database Management System (RDBMS).
- Till now, there is hardly any commercial product that follows all the 13 Codd's rules.
- Even **Oracle** follows only eight and half out (8.5) of 13.

Rule 0 (zero)

- This rule states that for a system to qualify as an **RDBMS**, it must be able to manage database entirely through the relational capabilities.
- All subsequent rules are based on the notation that in order for a database to be considered relational.

Rule 1 : Information Rule

- All information (including metadata – table names, column names and data types) is to be represented in table form as stored data in cells of tables.
- This is achieved by values in column and rows of tables.
- The rows and columns have to be strictly unordered.
- The basic requirement of the relational model.

Course_Code	Course_Title	*Course_Abbr*
22316	*Object Oriented Programming Using C++*	*OOP*
22317	Data Structure Using 'C'	DSU
22318	Computer Graphics	CGR
22319	Database Management System	*DMS*
22320	*Digital Techniques*	*DTE*

Rule 2 : Guaranteed Access Rule

- Each unique piece of data (atomic value) should be accessible by: **Table Name + primary key (Row) + Attribute (column name)**.
- All data are uniquely identified and accessible via this identity.

Primary Key

Course_Code	Course_Title	*Course_Abbr*
22316	*Object Oriented Programming Using C++*	*OOP*
22317	Data Structure Using 'C'	DSU
22318	Computer Graphics	CGR
22319	Database Management System	*DMS*
22320	*Digital Techniques*	*DTE*

Rule 3 : Systematic Treatment of NULL Values

- Null values are supported in fully relational DBMS for **representing missing information and inapplicable information** in a systematic way, independent of data type.
- A field should be allowed to remain empty.
- **Null** has several meanings: **missing data**, **not applicable** or **no value**.
- It should be handled consistently – **Not zero** or **Blank**.
- Primary key must not be null.
- Expression on **NULL** must give null.
- This involves the support of a null value, which is distinct from an empty string or number with a value of zero.

Course_Code	Course_Title	*Course_Abbr*	*Unit*	
22316	*Object Oriented Programming Using C++*	*OOP*	*5*	
22317	Data Structure Using 'C'	DSU	-	← Not null/value
22318	Computer Graphics	CGR	*5*	
22319	Database Management System	*DMS*	*5*	
22320	*Digital Techniques*	*DTE*		← **Null value**

Rule 4 : Active Online Catalog based on the Relational model

- Database dictionary (catalog) must have description of **Database**.
- Catalog to be governed by same rule as rest of the database.
- The system must support an online, inline, relational catalog that is accessible to authorized users by means of their regular query language.
- That is, users must be able to access the database's structure (catalog) using the same query language (i.e. SQL) that they use to access the database's data.

Rule 5 : Comprehensive Data Sub language Rule

- The Relational database must support at least one clearly defined language that includes comprehensive functionality for
    1. Data Definition (create, alter, drop, truncate, rename)
    2. View Definition
    3. Data Manipulation (insert, update, delete)
    4. Integrity Constraints (primary key, foreign key, null values)
    5. Authorization (Grant, Revoke)
    6. Transaction Control (begin, commit, rollback, savepoint)

Example:

- All commercial relational database use forms of standard **SQL** (**S**tructured **Q**uery **L**anguage) as their supported comprehensive language.
- SQL support
    - creating tables / views/ constraints/indexes,
    - accessing the records of tables/views (SELECT),
    - manipulating the records by insert/delete/update,
    - provides security by giving different level of access rights (GRANT and REVOKE) and integrity and consistency by using constraints

Rule 6 : View Updating Rule

- View is ”Virtual table”, derived from base tables.
- Views are subset of table / tables.
- Each view should support the same full range of data manipulation that has direct access to a table available.
- If a view is formed as join of 3 tables, changes to view should be reflected in base tables.
- In practice, providing update and delete access to logical view is difficult and not fully supported by any current database.
- It allow the update of simple theoretically updatable views, but disallow attempts to update complex views.

Rule 6 : View Updating Rule

- Below is a base table named ‘Course’.
- We can create a view by using some selected attributes.

Course_Code	Course_Title	Course_Abbr	Unit
22316	*Object Oriented Programming Using C++*	*OOP*	*5*
22317	Data Structure Using ‘C’	DSU	-
22318	Computer Graphics	CGR	*5*
22319	Database Management System	*DMS*	*5*

- To see(view) only course_code and course_title of above table we can use following syntax:

```
create view course_vu
as select course_code, course_title from course;
```

View is created.

```
Select * from course_vu;
```

Course_Code	Course_Title
22316	*Object Oriented Programming Using C++*
22317	Data Structure Using ‘C’
22318	Computer Graphics
22319	Database Management System

Rule 7 : High-level Insert, Update and Delete

- There must be Insert, Delete, and Update operations at each level of relations. Set operation like Union, Intersection and minus should also be supported.
- This rule states that insert, update, and delete operations should be supported for any retrievable set rather than just for a single row in a single table.
- All these operation should not be restricted to single table or row at a time. It should be able to handle multiple tables and rows in its operation

EXAMPLE:

- Suppose if we need to change **Course_code** then it will reflect everywhere automatic.
- Suppose employees get 5% increment in salary every year. Hence, the query should not be written for updating the salary one by one for thousands of employee. A single query should be strong enough to update the entire employee's salary at a time.

Rule 8 : Physical Data Independence

- **The ability to change the physical schema without changing the logical schema is called physical data independence.**
- The physical storage of data should not matter to the system. If say, some file supporting table were renamed or moved from one disk to another, it should not affect the application.
- The user is isolated from the physical method of storing and retrieving information from the database.
- Changes can be made to the underlying architecture (hardware, disk storage methods) without affecting how the user accesses it.

EXAMPLE:

- The user is isolated (Separated) from the physical method of storing and retrieving information from the database in which affecting directly in database.

Rule 9 : Logical Data Independence

- **The ability to change the logical (conceptual) schema without changing the External schema (User View) is called logical data independence.**
- If there is change in the logical structure (table structures) of the database the user view of data should not change.
- If a table is split into two tables, a new view should give result as the join of the two tables. This rule is most difficult to satisfy.

EXAMPLE:

- If we split the COURSE table according to department into multiple **course** tables, the user viewing the **course** table should not feel that these records are coming from different tables.
- These split tables should be able to get joined and show the result.
- In our example we can use UNION and display the results to the user.

Rule 10 : Integrity Independence

- Data integrity refers to maintaining assuring the accuracy and consistency of data over its entire life cycle.
- The database should be able to enforce its own integrity rather than using other programs.
- Key and Check constraints, trigger etc should be stored in Data Dictionary. This also makes **RDBMS** independent of front-end.
- The database language (like SQL) should support constraints on user input that maintain database integrity.
- This rule is not fully implemented by most major vendors.
- At a minimum, all databases do preserve two constraints through SQL.
  - No component of a primary key can have a null value.
  - If a foreign key is defined in one table, any value in it must exist as a primary key in another table.

Rule 11 : Distribution Independence

- A database should work properly regardless of its distribution across a network. This lays foundation of distributed database.
- Distribution independence implies that user should be totally unaware of whether or not the database is distributed (whether parts of database exist in multiple locations).
- He should be able to get the records as if he is pulling the records locally.
- Even if the database is located in different servers, the accessibility time should be comparatively less

Rule 12 : Non-Subversion Rule

- If the system provides a low-level (record-at-a-time) interface, then that interface cannot be used to subvert the system or bypass the integrity rule to change the data.
- This can be achieved by some sort of locking or encryption.

Key Concepts in RDBMS

- **KEYS in DBMS** is an attribute or set of attributes which helps us to identify a row(tuple) in a relation(table).
- They allow us to find the relation between two tables.
- Keys help us to uniquely identify a row in a table by a combination of one or more columns in that table.

EmployeeID	**FirstName**	**LastName**
11	Mohammad	Ali
22	Kousar	Ayub
33	Mohammed	Zaid
44	Shafaque	Mohsin

- In the above-given table, **EmployeeID** is a primary key because it uniquely identifies an employee record. In this table, no other employee can have the same employee ID.

Super Key

- A super key is a group of single or multiple keys which identifies rows in a table.
- A Super key may have additional attributes that are not needed for unique identification.
- **Example:**

Adharno	EmpNum	Empname
98123450988	MHSSP01	ALI
98765123456	MHSSP02	KOUSAR
19993789066	MHSSP03	ZAID
40325698743	MHSSP04	SHAFAQUE

- In the above-given example, Adharno and EmpNum are Super keys.

Candidate Key

- **CANDIDATE KEY** is a set of attributes that uniquely identify tuples in a table.
- Candidate Key is a super key with no repeated attributes.
- The Primary key should be selected from the candidate keys.
- Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only a single primary key.

Properties of Candidate key:

- It must contain unique values
- Candidate key may have multiple attributes
- Must not contain null values
- It should contain minimum fields to ensure uniqueness
- Uniquely identify each record in a table

Example: In the given table Stud ID, Roll No, and email are candidate keys which help us to uniquely identify the student record in the table

Stud ID	RollNo	FirstName	LastName	Email
1	11	Hatim	Mullajiwala	hattu@gmail.com
2	12	Habban	Bhoira	habbu@gmail.com
3	13	Aditya	Choudhary	addu@yahoo.com

Primary Key

- **PRIMARY KEY** is a column or group of columns in a table that uniquely identify every row in that table.
- The Primary Key can't be a duplicate meaning the same value can't appear more than once in the table.
- A table cannot have more than one primary key.
- **Rules for defining Primary key:**
  - Two rows can't have the same primary key value
  - It must for every row to have a primary key value.
  - The primary key field cannot be null.
  - The value in a primary key column can never be modified or updated if any foreign key refers to that primary key.
- **Example:**
  - In the following example, StudID is a Primary Key.

Stud ID	RollNo	FirstName	LastName	Email
1	11	Hatim	Mullajiwala	hattu@gmail.com
2	12	Habban	Bhoira	habbu@gmail.com
3	13	Aditya	Choudhary	addu@yahoo.com

Alternate Key

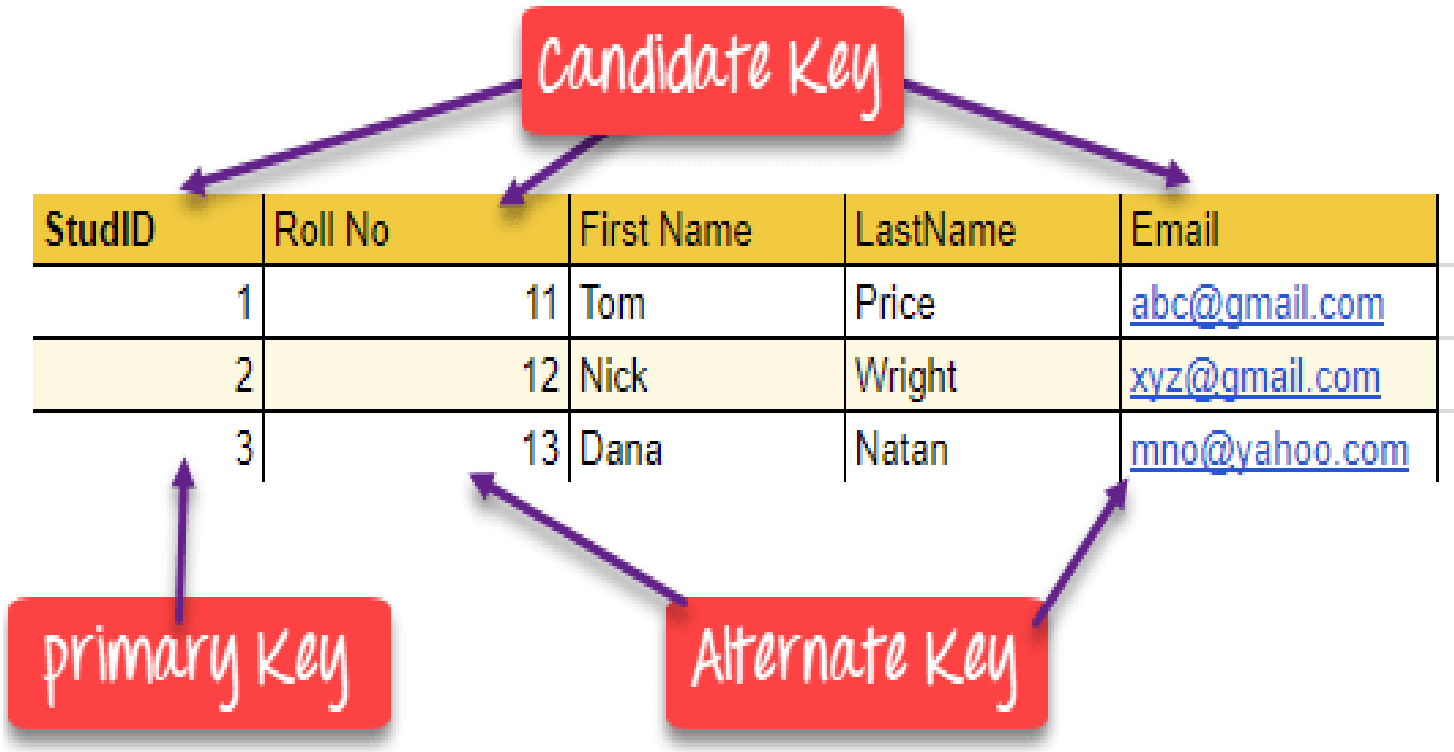
- **ALTERNATE KEYS** is a column or group of columns in a table that uniquely identify every row in that table.
- A table can have multiple choices for a primary key but only one can be set as the primary key.
- All the keys which are not primary key are called an Alternate Key.

Example:

- In this table, StudID, Roll No, Email are qualified to become a primary key.
- But since StudID is the primary key, Roll No & **Email** becomes the alternative key.

Stud ID	RollNo	FirstName	LastName	Email
1	11	Hatim	Mullajiwala	hattu@gmail.com
2	12	Habban	Bhoira	habbu@gmail.com
3	13	Aditya	Choudhary	addu@yahoo.com

Key Concepts in RDBMS



Foreign Key

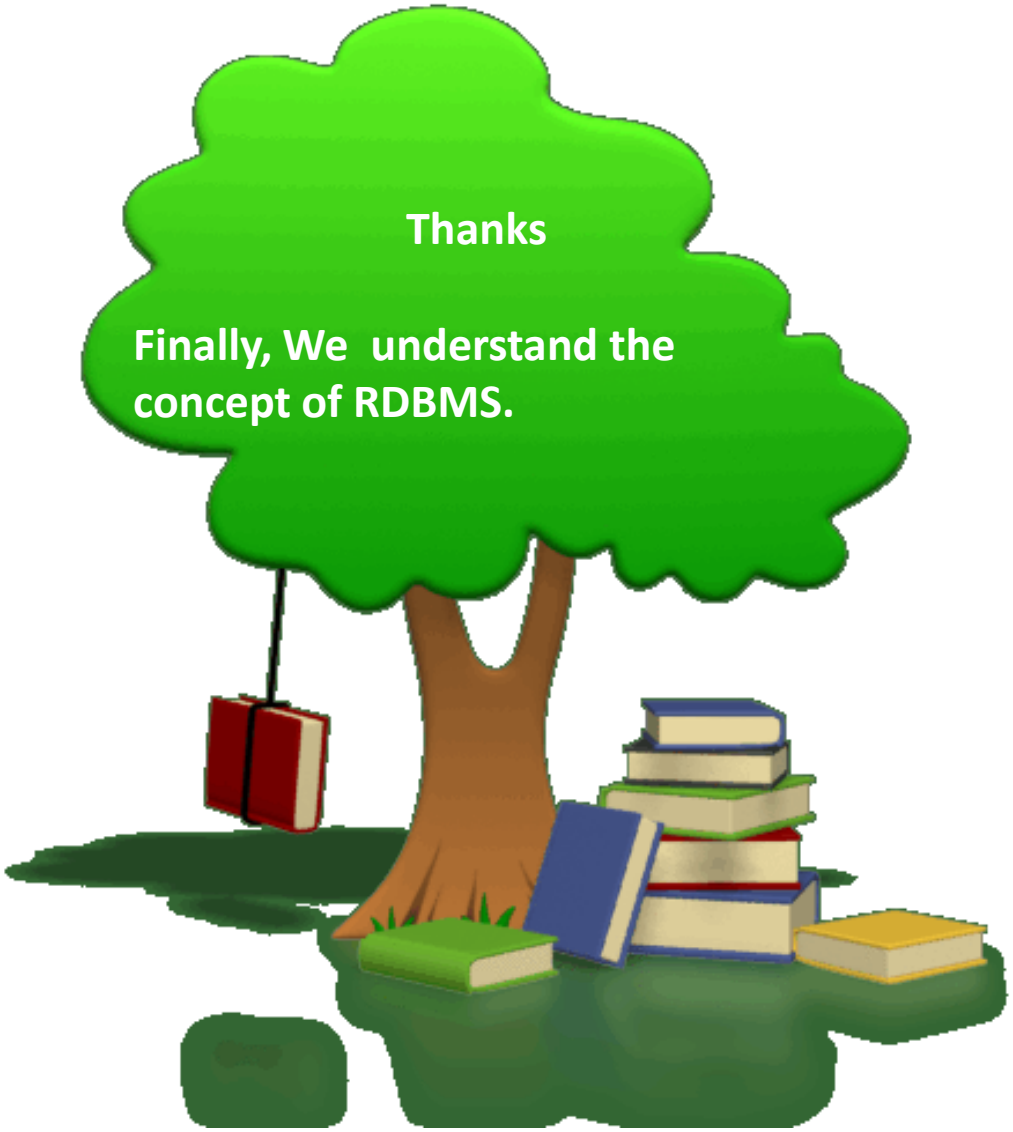
- **FOREIGN KEY** is a column that creates a relationship between two tables.
- The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity.
- It acts as a cross-reference between two tables as it references the primary key of another table.
- This concept is also known as Referential Integrity.

DeptCode	DeptName
001	Computer
002	Humanities & Science
003	Electrical

Teacher ID	Fname	Lname
T001	Mohammad	Ali
T002	Tahseen	Khan
T005	Mohammed	Zaid

- In this key example, we have two table, teacher and department in a school. However, there is no way to see which teacher work in which department.
- In this table, adding the foreign key in Deptcode to the Teacher name, we can create a relationship between the two tables.

Teacher ID	DeptCode	Fname	Lname
T001	001	Mohammad	Ali
T002	002	Tahseen	Khan
T005	001	Mohammed	Zaid


Thanks
Finally, We understand the concept of RDBMS.